



ZK DeFi Protocol

Proof of Achievement	
Proposal ID	1300196
Link	Project Catalyst Proposal
Milestone	2
Title	Smart Contract Development
Version	2.0 <u>Changes:</u> - added new section "Aiken Benchmarks" at the end
Status	Submitted - June 3rd 2025
Contributors	Dominik Tilman
Video	https://www.loom.com/share/13d2b3dc1c044d2f83da705d6217f67b?sid=5cd2e351-9192-47a9-a6f5-5b20051fb02b
Milestone Acceptance Criteria	1. Functional Smart contracts must allow for collateral verification, loan repayment, interest payment and loan issuance. 2. Smart contracts must be a. tested and verifiable on testnet b. run in reasonable time 3. Documentation and Flowcharts must explain each Smart Contracts in a comprehensible manner 4. Video must demonstrate milestone results in a comprehensible manner → Link: https://milestones.projectcatalyst.io/projects/1300196/milestones
Github	https://github.com/TrustLevel/ZK-Defi-Protocol

Onchain Components - Overview

1. Collateral Management Contract

This contract, deployed at

[addr_test1xzlcsxu3klwdr05wn2h3gy2cjlwvpqhgsfx9lvyl24h3ualsdperd7u6xlgax40zsg4397uczpw3qjvt7cge74t0reset8hra](#), demonstrates collateral management capabilities:

Core Functions:

- **Collateral Locking:** Securely holds user assets (ADA in this demonstration) until loan conditions are met
- **Ownership Tracking:** Maintains immutable records of collateral ownership through datums
- **State Management:** Implements a state machine with multiple states (unlocked, loan requested, loan processed, repayment requested)
- **Conditional Release:** Releases collateral only upon valid loan repayment or cancellation

DeFi Relevance:

- This contract implements a fundamental building block for any collateralized lending protocol, stablecoin system, or derivatives platform
- The state machine design pattern demonstrated here is essential for complex financial applications requiring multi-step processes

2. Issuance & Redemption Contract

The lending pool functions as an issuance and redemption system at

`addr_test1xpzcmt24d63y3ndp5sczngr82ka95ddcyh9na0q95f9g2z293xk42m4zfrx6rfps9x3xw4d6tg6msfw867qtgj2s5yss3udz5`):

Core Functions:

- **Token Issuance:** Issues borrowed assets (tUSDM tokens) against validated collateral
- **Redemption Processing:** Accepts repayments with interest and releases collateral claims
- **Parameter Management:** Maintains system parameters (collateralization ratios, interest rates)
- **Token Accounting:** Tracks total circulating supply and available liquidity

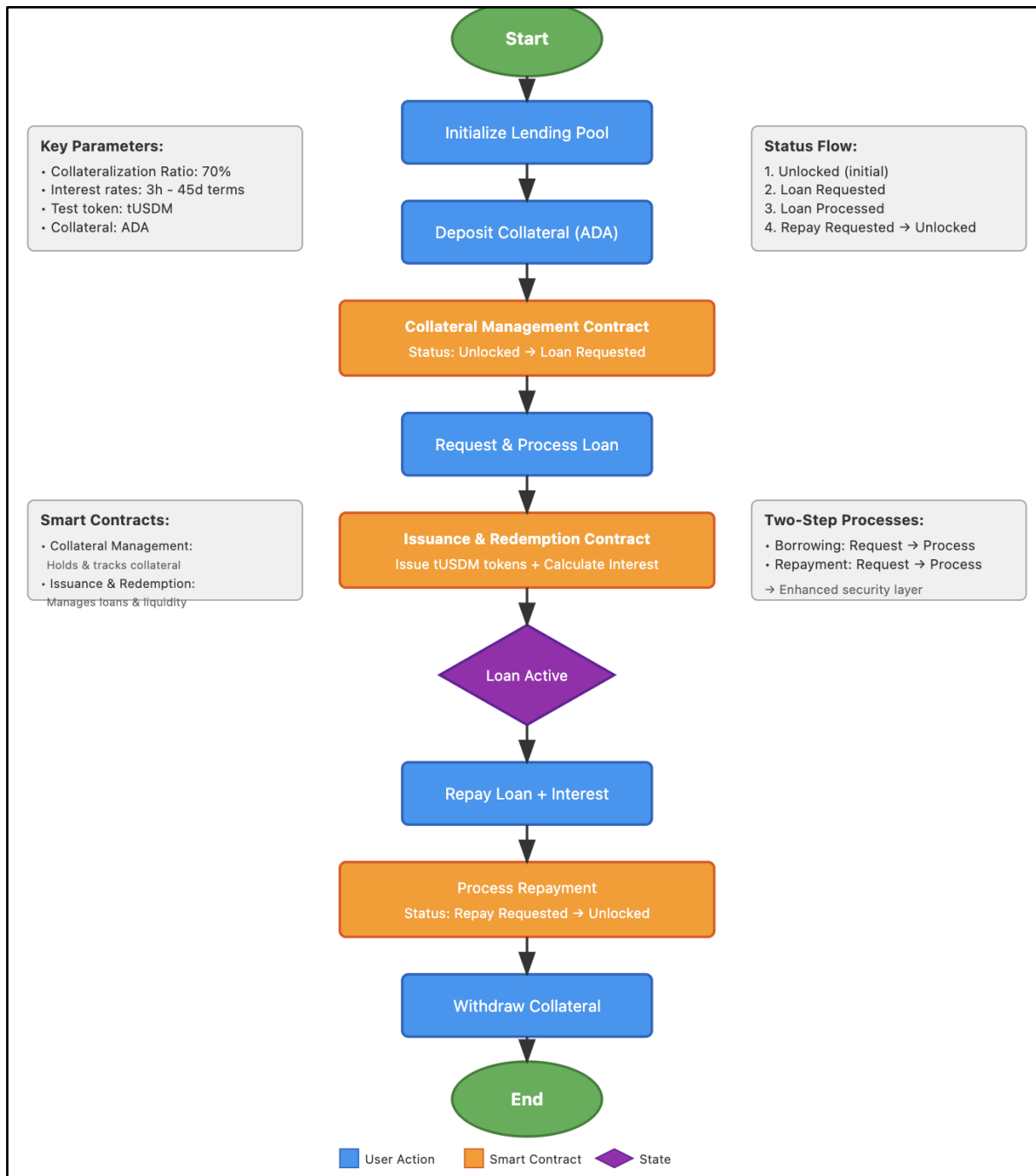
DeFi Relevance:

- This contract demonstrates principles essential for stablecoin issuance, synthetic asset creation, and tokenized real-world assets
- The interest calculation mechanism showcases time-based financial instruments

Onchain Logic - Flowchart

The demonstration showcases a complete lending cycle with the two smart contracts:

1. Depositing loanable assets to the lending pool
2. Depositing collateral
3. Borrowing against collateral
4. Repaying the loan with interest
5. Withdrawing unlocked collateral



Step-by-Step Demonstration with Blockchain Verification

This guide explains for each step how to verify our demonstration of the on-chain logic on the Cardano blockchain (Pre-prod).

The manual for the demonstration can be found here: <https://github.com/TrustLevel/ZK-Defi-Protocol/tree/main/offchain-module>

1. Initialize Lending Pool with Loanable Assets

Transaction: `a31ff0d18c6ee29009d6143b70db30dc88bcec64dfe41b202766bafb0a7a7433`

What Happened: This transaction initialized the lending pool by depositing tUSDM tokens (a test token representing USDM stablecoin). The protocol's initial settings were established, including:

- Collateralization ratio: 70% (users can borrow up to 70% of their collateral value)
- Collateral price: 750000 lovelace with 6 decimal places
- Interest rates for various loan terms (3 hours to 45 days)

How to Verify:

- Visit [Cardano Explorer](#)
- Observe that 10,000,000,000,000 tUSDM tokens were sent to the lending pool address
`addr_test1xpxcnt24d63y3ndp5sczngr82ka95ddcyh9na0q95f9g2z293xk42m4zfrx6rfps9x3xw4d6tg6msfwt867qtgj2s5yss3udz5`

2. Deposit Collateral

Transaction: `bc36bfb32a448cbe676731184292aa101f86f895c78a0fda154c0e136a4814b1`

What Happened: The user deposited 3,000 ADA as collateral. This created a UTXO at the collateral contract address with a datum indicating:

- The owner of the collateral (user address)
- The collateral's status (unused/unlocked)

How to Verify:

- Visit [Cardano Explorer](#)
- Observe that 3,000 ADA was sent to the collateral contract address
`addr_test1xzlcxsu3klwdr05wn2h3gy2cjlwvpqhgsfx9lvyvl24h3ualsdperd7u6xlgax40zsg4397uczpw3qjvt7cge74t0reset8hra`

3. Borrow Against Collateral (Two-Step Process)

a. User's Loan Request

Transaction: `ff16455c356a835db45c5b2c1502764c121d748a300c87001e8bf2e0069bd256`

What Happened: The user submitted a loan request, updating the collateral's datum to indicate:

- Loan status: "LoanRequested"
- Borrowed asset: tUSDM
- Borrowed amount: 525,000,000 (525 tUSDM)
- Loan term: 10,800,000 ms (3 hours)

How to Verify:

- Visit [Cardano Explorer](#)
- The transaction updates the collateral UTXO with new datum information

b. Admin Processes Loan

Transaction: [3789defcbe736ab0c606220b7c387d14da864de5cbf0e3cc2d1b75007bc2588](#)

What Happened: The admin/protocol fulfilled the loan request:

- Sent 525 tUSDM to the user
- Updated the collateral datum to "LoanProcessed" status
- Added calculated interest (26.25 tUSDM)
- Set loan maturity date

How to Verify:

- Visit [Cardano Explorer](#)
- Observe that 525 tUSDM tokens were transferred to the user's address
- The collateral UTXO was updated with new status

4. Repay Loan (Two-Step Process)

a. User's Repayment Request

Transaction: [d8ef72624b20b829e7dca48a56c80e9d0b79c6908c43ea1040549a5023f0d1dc](#)

What Happened: The user submitted a repayment request:

- Sent 551.25 tUSDM (principal + interest) to the collateral UTXO
- Updated the collateral datum to "RepayRequested" status

How to Verify:

- Visit [Cardano Explorer](#)
- Observe that the collateral UTXO now contains both ADA and tUSDM tokens
- The datum status has changed to indicate repayment

b. Admin Processes Repayment

Transaction: [7d07900206903f4045f849ceb6aaa0c2a5049b4e080a6094fd3ee6f8d9e53c49](#)

What Happened: The admin/protocol processed the repayment:

- Returned the repaid amount (principal + interest) to the lending pool
- Updated the collateral UTXO back to "unlocked" status

How to Verify:

- Visit [Cardano Explorer](#)
- The lending pool balance is increased by 551.25 tUSDM

- The collateral UTXO is reset to its initial state (unlocked)

5. Withdraw Collateral

Transaction: `28d6f87901d1046a5d18e3a6a510b60f04eae323b08463f2edaf673a63384d3a`

What Happened: The user withdrew their unlocked collateral:

- 3,000 ADA was returned to the user's address
- The collateral UTXO was consumed

How to Verify:

- Visit [Cardano Explorer](#)
- Observe that 2,999.522713 ADA (original amount minus transaction fees) was sent back to the user's wallet

Key Addresses in the Protocol

1. Issuance & Redemption Contract:

`addr_test1xpczcnt24d63y3ndp5scznngn82ka95ddcyh9na0q95f9g2z293xk42m4zfrx6rfps9x3xw4d6tg6msfwt867qtgj2s5yss3udz5`

- Manages the loanable assets and protocol settings
- Holds tUSDM tokens available for borrowing

2. Collateral Management Contract:

`addr_test1xzlcxsu3klwdr05wn2h3gy2cjlwvpqhgsfx9lvyvl24h3ualsdperd7u6xlgax40zsg4397uczpw3qjvt7cge74t0reset8hra`

- Holds user collateral (ADA in this case)
- Tracks loan status via datums

3. Admin Address:

`addr_test1qr7pczj8y85kux5zwe3u64mgun2vnn8rcxmcmkgkhndnzwy2tuma0zhatn6rc6llml7l0876t59z4qxl4w56k8058mqxcp45h`

- Manages the protocol operations
- Holds the beacon tokens used for protocol identification

Aiken Testcases

1. Collateral Management Contract:

```
dominiktilman@Dominiks-Laptop onchain-module % aiken check
Compiling trustlevel/lending 0.0.2 (.)
Compiling aiken-lang/stdlib v2.2.0 (./build/packages/aiken-lang-stdlib)
Testing ...

collateral
PASS [mem: 63698, cpu: 19156807] withdraw_succeed
PASS [mem: 67677, cpu: 19534446] withdraw_fail
  • with traces
    | collateral_not_locked ? False
PASS [mem: 296045, cpu: 95576310] borrow_req_succeed
PASS [mem: 320114, cpu: 96867403] borrow_req_fail
  • with traces
    | values_match ? False
    | collateral_output_valid ? False
PASS [mem: 246532, cpu: 71732903] borrow_proc_succeed
PASS [mem: 254766, cpu: 73256499] borrow_proc_fail
  • with traces
    | amt_in_pool > loan_amt ? False
PASS [mem: 332576, cpu: 108006620] repay_req_succeed
PASS [mem: 369644, cpu: 114254437] repay_req_fail
  • with traces
    | collateral_output_valid ? False
PASS [mem: 743487, cpu: 229252905] repay_proc_succeed
  • with traces
    | input_lovelace : 1000000000
    | output_lovelace : 1000000000
    | input_collateral_amt : 1000000000
    | output_collateral_amt : 1000000000
PASS [mem: 791761, cpu: 243378809] repay_proc_fail
  • with traces
    | input_lovelace : 1000000000
    | output_lovelace : 1000000000
    | input_collateral_amt : 1000000000
    | output_collateral_amt : 1000000000
    | output_datum == expected_datum ? False
    | collateral_output_valid ? False
10 tests | 10 passed | 0 failed
```

2. Lending Pool Contract:

```
lending_pool
PASS [mem: 3635878, cpu: 1128697123] borrow_proc_succeed
- with traces
| Expected:
| owner : 121([_ 121([_ h'D48838FEE1E840857DAB28A04DD88193AEC8C297B85D8150C0A1F5CC']), 121([_ 121([_ 121([_
h'6313A1D2C296EB3341E159B6C5C6991DE11E81062B95108C9AA024AD'])))])
| status : 122([])
| borrowed_asset : 121([_ h'C48CBB3D5E57ED56E276BC45F99AB39ABE94E6CD7AC39FB402DA47AD', h'0014DF105553444D'])
| borrowed_amt : 525000000
| interest_amt : 131250000
| loan_term : 3888000000
| validTo : 1739793600000
| maturity : 1743681600000
| Actual:
| owner : 121([_ 121([_ h'D48838FEE1E840857DAB28A04DD88193AEC8C297B85D8150C0A1F5CC']), 121([_ 121([_ 121([_
h'6313A1D2C296EB3341E159B6C5C6991DE11E81062B95108C9AA024AD'])))])
| status : 122([])
| borrowed_asset : 121([_ h'C48CBB3D5E57ED56E276BC45F99AB39ABE94E6CD7AC39FB402DA47AD', h'0014DF105553444D'])
| borrowed_amt : 525000000
| interest_amt : 131250000
| loan_term : 3888000000
| validTo : 1739793600000
| maturity : 1743681600000
| reserve_amt_input : 1000000000000
| reserve_amt_output : 999475000000
| expected_reserve_amt_output: 999475000000
PASS [mem: 3587342, cpu: 1116508076] borrow_proc_fail
- with traces
| Expected:
| owner : 121([_ 121([_ h'D48838FEE1E840857DAB28A04DD88193AEC8C297B85D8150C0A1F5CC']), 121([_ 121([_ 121([_
h'6313A1D2C296EB3341E159B6C5C6991DE11E81062B95108C9AA024AD'])))])
| status : 122([])
| borrowed_asset : 121([_ h'C48CBB3D5E57ED56E276BC45F99AB39ABE94E6CD7AC39FB402DA47AD', h'0014DF105553444D'])
| borrowed_amt : 525000000
| interest_amt : 131250000
| loan_term : 3888000000
| validTo : 1739793600000
| maturity : 1743681600000
| Actual:
| owner : 121([_ 121([_ h'D48838FEE1E840857DAB28A04DD88193AEC8C297B85D8150C0A1F5CC']), 121([_ 121([_ 121([_
h'6313A1D2C296EB3341E159B6C5C6991DE11E81062B95108C9AA024AD'])))])
| status : 122([])
| borrowed_asset : 121([_ h'C48CBB3D5E57ED56E276BC45F99AB39ABE94E6CD7AC39FB402DA47AD', h'0014DF105553444D'])
| borrowed_amt : 525000000
| interest_amt : 131250000
| loan_term : 3888000000
| validTo : 1739793600000
| maturity : 1743681600000
```

```

| Actual:
| owner : 121([_ 121([_ h'D48838FEE1E840857DAB28A04DD88193AEC8C297B85D8150C0A1F5CC']), 121([_ 121([_ 121([_
h'6313A1D2C296EB3341E159B6C5C6991DE11E81062B95108C9AA024AD')]))]))))
| status : 122([[]])
| borrowed_asset : 121([_ h'C48CBB3D5E57ED56E276BC45F99AB39ABE94E6CD7AC39FB402DA47AD', h'0014DF105553444D'])
| borrowed_amt : 525000000
| interest_amt : 131250000
| loan_term : 3888000000
| validTo : 1739793600000
| maturity : 1743681600000
|
| reserve_amt_input      : 1000000000000
| reserve_amt_output     : 999474999999
| expected_reserve_amt_output: 999475000000
| values_match ? False
| lending_pool_output_valid ? False
PASS [mem: 928939, cpu: 297972727] repay_proc_succeed
• with traces
| repayment_time : 174368159998
| loan.maturity : 1743681600000
| reserve_amt_input      : 1000000000000
| reserve_amt_output     : 1000656250000
| expected_reserve_amt_output: 1000656250000
PASS [mem: 920211, cpu: 291248907] repay_proc_fail
• with traces
| repayment_time : 1743681600001
| loan.maturity : 1743681600000
| reserve_amt_input      : 1000000000000
| reserve_amt_output     : 1000656249999
| expected_reserve_amt_output: 1000656250000
| values_match ? False
| lending_pool_output_valid ? False

```

4 tests | 4 passed | 0 failed

Aiken Benchmarks

<https://aiken-lang.org/language-tour/bench>

To include benchmarks (part of the latest Aiken update 1.1.15) we had to upgrade our Aiken version (previous 1.1.9) and import the module “aiken/fuzz”.

To keep the repository and the code clean, we created 2 separate validator scripts (collateral.ak → bench_collateral.ak; lending_pool.ak → bench_lending_pool.ak) that include the original code plus 3 benchmarks for each validator. The files can be found here:

[Benchmark Validator Files](#)

1. Benchmark Results collateral.ak:

- Memory: 31,550 → 510,108 (steep scaling)
- CPU: 9M → 167M (steep scaling)

Insights:

1. **Withdraw runs at very low costs!**
2. **BorrowRequest is more expensive than Withdraw:**
 - ~5x more Memory usage
 - ~5x more CPU usage
 - But: Costs remain constant (no scaling with loan_size)
3. **Output Processing has steeper scaling than Withdraw:**
 - 16x Memory increase (31K → 510K)
 - 18x CPU increase (9M → 167M)
 - This shows that `get_protocol_outputs` becomes more expensive with many outputs, but still at very moderate levels.
4. **All operations stay well within limits:**
 - Max Memory: 510KB of 14MB  (3.6%)
 - Max CPU: 167M of 10,000M  (1.7%)

→ Why is BorrowRequest (2nd graph) constant?

The `loan_size` only affects the value, not the complexity of the operation. The actual work (validation, checks) remains the same. This is good - it means large loans aren't more expensive than small ones!

→ **Summary: This validator performs very efficiently and can handle significant scaling without approaching transaction limits.**

2. Benchmark Results lending_pool.ak:



1. benchmark_collateral_calculation

- Memory: **32,745** (constant)
- CPU: **10,344,572** (constant)
- **Meaning:** Mathematical calculation for collateral has fixed costs, regardless of amount

2. benchmark_validity_range

- Memory: **19,791** (constant)
- CPU: **6,279,406** (constant)
- **Meaning:** Time validation is very cheap with constant costs

3. benchmark_borrow_process

- Memory: **823,145** (constant)
- CPU: **274,697,824** (constant)
- **Meaning:** The complete borrow process is the most expensive operation

Insights:

1. **Collateral calculation runs very efficiently with low costs.**
2. **BorrowProcess is the most expensive aspect of the protocol:**
 - ~25x more Memory than collateral calculation
 - ~27x more CPU than collateral calculation
 - Almost **275M CPU Units** (but still only 2.75% of the limit!)
3. **Lending Pool is more expensive than Collateral Validator:**
 - Collateral **BorrowRequest**: 87M CPU
 - Lending Pool **BorrowProcess**: 275M CPU
 - This makes sense - Lending Pool does more work
4. **All operations stay well within limits:**
 - Max Memory: 823KB of 14MB  (5.9%)
 - Max CPU: 275M of 10,000M  (2.75%)

→ Summary: This validator performs very efficiently and the constant graphs show that costs don't scale with transaction size - this is good for cost predictability!